

OCP Maintenance AI — Diagramas de Arquitectura

Generado: 2026-03-15 | Stack: FastAPI + PostgreSQL + React 19 + Docker
Total: 20 routers, 28 services, 49 tablas DB, 4 agentes IA, 126+ herramientas, 25 paginas frontend

1. Diagrama UML de Componentes

```
graph TB
    subgraph Cliente
        REACT[React 19 + Vite 7]
        SW[Service Worker PWA]
        IDB[IndexeDB Offline]
    end

    subgraph Nginx
        SSL[Nginx :8080/:8443 - SSL Termination]
    end

    subgraph Backend
        MW[Middleware: Security + RateLimit + CORS]
        ROUTERS[20 API Routers]
        SERVICES[28 Services]
        CORTEX[CORTEX AI Engine]
    end

    subgraph Database
        PG[(PostgreSQL 16 - 49 tablas)]
    end

    subgraph APIs_Externas
        ANTHROPIC[Anthropic Claude API]
        WHISPER[OpenAI Whisper API]
    end

    REACT --> SSL
    SSL --> MW
    MW --> ROUTERS
    ROUTERS --> SERVICES
    ROUTERS --> CORTEX
    SERVICES --> PG
    CORTEX --> ANTHROPIC
    ROUTERS --> WHISPER
    REACT --> SW
    REACT --> IDB
```

2. Diagrama de Componentes Detallado — Backend

```
graph LR
    subgraph Routers_Campo
        R1["capture"]
        R2["transcribe"]
        R3["work-requests"]
    end

    subgraph Routers_Planificacion
        R4["planner"]
        R5["backlog"]
        R6["scheduling"]
        R7["calendar"]
    end

    subgraph Routers_Ingenieria
        R8["hierarchy"]
        R9["criticality"]
        R10["fmea"]
        R11["tasks"]
        R12["work-packages"]
    end

    subgraph Routers_Confiabilidad
        R13["reliability"]
        R14["rca"]
        R15["analytics"]
    end

    subgraph Routers_Gestion
        R16["sap"]
        R17["reporting"]
        R18["dashboard"]
        R19["admin"]
    end

    subgraph Routers_AI
        R20["ai - CORTEX"]
        R21["auth"]
    end

    subgraph Servicios
        S1[hierarchy_service]
        S2[criticality_service]
        S3[fmea_service]
        S4[capture_service]
        S5[planner_service]
        S6[backlog_service]
        S7[scheduling_service]
        S8[reliability_service]
        S9[rca_service]
        S10[reporting_service]
        S11[sap_service]
        S12[analytics_service]
        S13[auth_service]
        S14[agent_service]
    end
```

```
R1 --> S4  
R3 --> S4  
R4 --> S5  
R5 --> S6  
R6 --> S7  
R8 --> S1  
R9 --> S2  
R10 --> S3  
R13 --> S8  
R14 --> S9  
R15 --> S12  
R16 --> S11  
R17 --> S10  
R20 --> S14  
R21 --> S13
```

3. BPMN — Proceso de Mantenimiento Completo

```
graph LR
    START((Inicio)) --> CAPTURE{Tipo captura?}
    CAPTURE -->|Voz| VOICE[Captura por Voz]
    CAPTURE -->|Texto| TEXT[Captura Texto]
    CAPTURE -->|Imagen| IMAGE[Foto + AI Vision]
    VOICE --> WR[Crear Work Request]
    TEXT --> WR
    IMAGE --> WR
    WR --> AICLASS[AI Clasifica y Prioriza]
    AICLASS --> DUP{Duplicado?}
    DUP -->|Si| MERGE[Fusionar]
    DUP -->|No| VALIDATE[Validar]
    MERGE --> VALIDATE
    VALIDATE --> PLANNER[Planner AI Recomendación]
    PLANNER --> ACTION{Acción?}
    ACTION -->|Aprobar| BACKLOG[Agregar a Backlog]
    ACTION -->|Modificar| PLANNER
    ACTION -->|Escalar| ESCALATE[Escalar a Gerencia]
    ACTION -->|Diferir| DEFER[Diferir]
    ESCALATE --> BACKLOG
    BACKLOG --> OPTIMIZE[Optimizar Backlog]
    OPTIMIZE --> WEEKLY[Programa Semanal]
    WEEKLY --> RESOURCE[Nivelar Recursos]
    RESOURCE --> CONFLICT{Conflictos?}
    CONFLICT -->|Si| RESOURCE
    CONFLICT -->|No| FINALIZE[Finalizar]
    FINALIZE --> GANTT[Generar Gantt]
    GANTT --> EXECUTE[Ejecutar Trabajo]
    EXECUTE --> COMPLETE[Completar Orden]
    COMPLETE --> KPI[Calcular KPIs]
    KPI --> RCAQ{Fallo repetitivo?}
    RCAQ -->|Si| RCA[Análisis RCA]
    RCAQ -->|No| REPORT[Reportes]
    RCA --> CAPA[Crear CAPA]
    CAPA --> REPORT
    REPORT --> ENDX((Fin))
```

4. BPMN — Proceso CORTEX AI Session (4 Milestones)

```
graph TB
    START((Usuario)) --> CREATE[POST /ai/sessions]
    CREATE --> M1[MILESTONE 1: Data Collection]
    M1 --> ORCH1[Orchestrator analiza equipo]
    ORCH1 --> DEL{Delegar?}
    DEL -->|Criticidad| REL[Reliability Agent]
    DEL -->|Repuestos| SPARE[Spare Parts Agent]
    DEL -->|Plan| PLAN[Planning Agent]
    REL --> G1{Gate 1 OK?}
    SPARE --> G1
    PLAN --> G1
    G1 -->|No| M1
    G1 -->|Si| M2[MILESTONE 2: Analysis]
    M2 --> FMECA[FMECA Analysis]
    FMECA --> RCM[RCM Decision]
    RCM --> G2{Gate 2 OK?}
    G2 -->|No| M2
    G2 -->|Si| M3[MILESTONE 3: Tasks]
    M3 --> TASKS[Generar Tareas]
    TASKS --> WP[Agrupar Work Packages]
    WP --> G3{Gate 3 OK?}
    G3 -->|No| M3
    G3 -->|Si| M4[MILESTONE 4: SAP]
    M4 --> SAPG[Generar Paquete SAP]
    SAPG --> VALIDS[Validar Campos]
    VALIDS --> EXPORT[Exportar XLSX]
    EXPORT --> ENDX((Completado))
```

5. Diagrama de Clases — Database Models (Core)

```
classDiagram
    PlantModel "1" --> "*" HierarchyNodeModel
    PlantModel "1" --> "*" UserModel
    HierarchyNodeModel "1" --> "*" HierarchyNodeModel
    HierarchyNodeModel "1" --> "*" CriticalityAssessmentModel
    HierarchyNodeModel "1" --> "*" FunctionModel
    FunctionModel "1" --> "*" FunctionalFailureModel
    FunctionalFailureModel "1" --> "*" FailureModeModel
    FailureModeModel "1" --> "*" MaintenanceTaskModel
    HierarchyNodeModel "1" --> "*" WorkPackageModel
    WorkPackageModel "*" --> "1" SAPUploadPackageModel
    FieldCaptureModel "1" --> "1" WorkRequestModel
    WorkRequestModel "1" --> "1" BacklogItemModel
    UserModel "1" --> "*" AISessionModel
    AISessionModel "1" --> "*" AIInteractionModel

class UserModel {
    user_id : UUID PK
    email : String UNIQUE
    username : String UNIQUE
    password_hash : String
    role : String
    plant_id : String FK
    is_active : Boolean
}

class PlantModel {
    plant_id : String PK
    name : String
    name_fr : String
    name_ar : String
    location : String
}

class HierarchyNodeModel {
    node_id : UUID PK
    parent_node_id : UUID FK
    level : Integer
    node_type : String
    code : String
    tag : String
    criticality : String
    plant_id : String FK
}

class CriticalityAssessmentModel {
    assessment_id : UUID PK
    node_id : UUID FK
    criteria_scores : JSON
    overall_score : Float
    risk_class : String
    status : String
}

class FunctionModel {
    function_id : UUID PK
```

```

        node_id : UUID FK
        function_type : String
        description : String
        ai_generated : Boolean
    }

    class FunctionalFailureModel {
        failure_id : UUID PK
        function_id : UUID FK
        failure_type : String
        description : String
    }

    class FailureModeModel {
        failure_mode_id : UUID PK
        functional_failure_id : UUID FK
        mechanism : String
        cause : String
        strategy_type : String
        ai_confidence : Float
    }

    class MaintenanceTaskModel {
        task_id : UUID PK
        failure_mode_id : UUID FK
        name : String
        task_type : String
        constraint : String
        frequency_value : Integer
        frequency_unit : String
    }

    class WorkPackageModel {
        work_package_id : UUID PK
        node_id : UUID FK
        code : String
        frequency_value : Integer
        constraint : String
        allocated_tasks : JSON
    }

    class SAPUploadPackageModel {
        package_id : UUID PK
        plant_code : String
        maintenance_plan : JSON
        task_lists : JSON
        status : String
    }

    class FieldCaptureModel {
        capture_id : UUID PK
        capture_type : String
        raw_text : String
        equipment_tag_manual : String
        language : String
        technician_id : String
    }

    class WorkRequestModel {
        work_request_id : UUID PK

```



```
    source_capture_id : UUID FK
    status : String
    equipment_id : String
    ai_classification : JSON
}

class BacklogItemModel {
    backlog_id : UUID PK
    work_request_id : UUID FK
    priority : Integer
    wo_type : String
    estimated_hours : Float
    shutdown_required : Boolean
}

class AISessionModel {
    session_id : UUID PK
    user_id : UUID FK
    equipment_tag : String
    status : String
    current_milestone : Integer
    milestone_gates : JSON
}

class AIInteractionModel {
    interaction_id : UUID PK
    session_id : UUID FK
    agent_type : String
    milestone : Integer
    tool_calls : JSON
    tokens_used : Integer
}
```

6. Diagrama de Clases — Models Complementarios

```
classDiagram
class RCAAnalysisModel {
    analysis_id : UUID PK
    event_description : String
    plant_id : String
    equipment_id : String
    level : String
    status : String
    cause_effect : JSON
    solutions : JSON
    analysis_5w2h : JSON
}

class WeeklyProgramModel {
    program_id : UUID PK
    plant_id : String
    week_number : Integer
    year : Integer
    status : String
    work_packages : JSON
    total_hours : Float
}

class TroubleshootingDiagnosticModel {
    diagnostic_id : UUID PK
    equipment_tag : String
    symptom_description : String
    ai_diagnosis : JSON
    probable_causes : JSON
    confidence_score : Float
    resolved : Boolean
}

class WorkOrderModel {
    work_order_id : String PK
    equipment_id : String
    order_type : String
    priority : String
    status : String
    actual_duration_hours : Float
}

class HealthScoreModel {
    score_id : UUID PK
    node_id : UUID FK
    composite_score : Float
    health_class : String
    trend : String
    recommendations : JSON
}

class KPIMetricsModel {
    metrics_id : UUID PK
    plant_id : String
    mtbf_days : Float
    mtrr_hours : Float
}
```

```
        availability_pct : Float
        oee_pct : Float
        pm_compliance_pct : Float
    }

    class ReportModel {
        report_id : UUID PK
        report_type : String
        plant_id : String
        period_start : Date
        period_end : Date
        content : JSON
    }

    class CAPAItemModel {
        capa_id : UUID PK
        capa_type : String
        title : String
        current_phase : String
        status : String
        actions_planned : JSON
    }

    class FMECAWorksheetModel {
        worksheet_id : UUID PK
        equipment_id : String
        status : String
        current_stage : String
        rows : JSON
    }
}
```

7. Diagrama de Despliegue

```
graph TB
    USER[Usuario Browser/PWA] -->|HTTPS| NGINX_S[Nginx :8080/:8443]

    subgraph Docker_Compose
        NGINX_S -->|static files| FRONTEND[React 19 Build - 74 assets]
        NGINX_S -->|/api/v1| BACKEND[FastAPI :8000]
        BACKEND --> GUNICORN[Gunicorn 4 workers]
        GUNICORN --> UVICORN[Uvicorn ASGI]
        BACKEND --> POSTGRES[(PostgreSQL 16)]
        BACKEND --> CORTEX_E[CORTEX AI Engine]
    end

    CORTEX_E --> ANTHROPIC_E[Anthropic Claude API]
    BACKEND --> WHISPER_E[OpenAI Whisper API]

    subgraph Config
        ENV[.env: JWT_SECRET, ANTHROPIC_API_KEY, DATABASE_URL]
        CERTS[./certs/ SSL Certificates]
    end

    ENV --> BACKEND
    CERTS --> NGINX_S
```

8. Diagrama de Secuencia — Captura de Campo a SAP

```
sequenceDiagram
    participant T as Tecnico
    participant FE as Frontend
    participant API as FastAPI
    participant DB as PostgreSQL
    participant AI as CORTEX

    T->>FE: Graba audio o toma foto
    FE->>API: POST /capture
    API->>DB: INSERT field_captures
    API->>AI: Clasificar captura
    AI-->>API: equipment_tag, priority
    API->>DB: INSERT work_requests
    API-->>FE: Work request creada

    T->>FE: Solicita recomendacion
    FE->>API: POST /planner/recommend
    API->>AI: Analizar WR + recursos
    AI-->>API: Recomendacion
    API->>DB: INSERT planner_recommendations
    API-->>FE: Recomendacion lista

    T->>FE: Aprueba
    FE->>API: PUT /planner/action
    API->>DB: INSERT backlog_items
    API-->>FE: En backlog

    T->>FE: Crear programa semanal
    FE->>API: POST /scheduling/programs
    API->>DB: INSERT weekly_programs
    API-->>FE: Programa + Gantt

    T->>FE: Inicia sesion CORTEX
    FE->>API: POST /ai/sessions
    API->>DB: INSERT ai_sessions
    loop 4 Milestones
        FE->>API: POST /ai/advance
        API->>AI: Ejecutar milestone
        AI-->>API: Resultado + tools
        API->>DB: INSERT ai_interactions
    end
    end
    API-->>FE: Sesion completada

    T->>FE: Exportar SAP
    FE->>API: POST /sap/generate-upload
    API->>DB: INSERT sap_packages
    FE->>API: GET /sap/export
    API-->>FE: Descarga XLSX
```

9. Diagrama de Estado — Work Request Lifecycle

```
stateDiagram-v2
    [*] --> CAPTURED
    CAPTURED --> CLASSIFIED
    CLASSIFIED --> VALIDATED
    VALIDATED --> APPROVED
    VALIDATED --> DEFERRED
    VALIDATED --> ESCALATED
    APPROVED --> IN_BACKLOG
    ESCALATED --> IN_BACKLOG
    IN_BACKLOG --> SCHEDULED
    SCHEDULED --> IN_PROGRESS
    IN_PROGRESS --> COMPLETED
    IN_PROGRESS --> ON_HOLD
    ON_HOLD --> IN_PROGRESS
    COMPLETED --> CLOSED
    DEFERRED --> VALIDATED
    CLOSED --> [*]
```

10. Diagrama C4 — Contexto del Sistema

```
graph TB
    TECH[Tecnico de Campo] --> FE
    PLANNER[Planificador] --> FE
    ENG[Ingeniero] --> FE
    MGR[Gerente] --> FE
    ADMN[Administrador] --> FE

    subgraph OCP_Maintenance_AI_Platform
        FE[Frontend React 19 + PWA]
        BE[Backend FastAPI + 28 services]
        CORTEX_P[CORTEX: 4 AI Agents + 126 Tools]
        DB_P[(PostgreSQL 49 tablas)]
        FE --> BE
        BE --> CORTEX_P
        BE --> DB_P
    end

    end

    CORTEX_P --> CLAUDE[Anthropic Claude API]
    BE --> WHISPERP[OpenAI Whisper API]
    BE --> SAP_P[SAP PM via XLSX]
```

11. CORTEX AI — Arquitectura de Agentes

```
graph TB
    USER_Q[Instruccion del Usuario] --> ORCH[Orchestrator Agent - Sonnet 4.5]

    ORCH -->|Delega criticidad| RELIABILITY[Reliability Agent - Opus]
    ORCH -->|Delega planificacion| PLANNING[Planning Agent - Sonnet]
    ORCH -->|Delega repuestos| SPAREPARTS[Spare Parts Agent - Haiku]

    RELIABILITY --> TOOLS_R[Tools: criticality, fmea, weibull, rca, rbi, jackknife]
    PLANNING --> TOOLS_P[Tools: backlog, scheduling, gantt, kpi, validation]
    SPAREPARTS --> TOOLS_S[Tools: materials, stock_levels, spare_analysis]

    ORCH --> SKILLS[Skills Loader: SOPs + Checklists]
    ORCH --> PROMPTS[System Prompts: MD files]

    subgraph Tool_Categories
        T1[Backlog Tools - 6]
        T2[CAPA Tools - 7]
        T3[Criticality Tools - 4]
        T4[FMECA Tools - 3]
        T5[KPI Tools - 5]
        T6[RCA Tools - 7]
        T7[RCM Tools - 2]
        T8[Reliability Tools - 7]
        T9[SAP Tools - 3]
        T10[Scheduling Tools - 7]
        T11[Validation Tools - 6]
        T12[Weibull Tools - 3]
        T13[Reporting Tools - 7]
        T14[+ 20 mas...]
    end

    end
```

12. Frontend — Mapa de Paginas por Rol

```
graph LR
    subgraph ALL_ROLES
        A1[Dashboard]
        A2[Work Requests]
        A3[Hierarchy]
        A4[Profile]
    end

    subgraph FIELD_ROLES
        F1[Field Capture]
    end

    subgraph PLAN_ROLES
        P1[Work Packages]
        P2[Backlog]
        P3[Scheduling]
        P4[Planner]
        P5[Planning]
        P6[Criticality]
        P7[FMEA]
        P8[FMECA]
        P9[Strategy]
    end

    subgraph ENGR_ROLES
        E1[Reliability]
        E2[RCA]
        E3[Defect Elimination]
        E4[AI Agents CORTEX]
        E5[Troubleshooting]
    end

    subgraph MGMT_ROLES
        M1[Analytics]
        M2[Executive Dashboard]
        M3[Reports]
        M4[SAP Review]
    end

    subgraph ADMIN_ONLY
        AD1[Admin Panel]
    end
```


Resumen de Metricas

Metrica	Valor
Routers API	20
Endpoints totales	~130
Servicios backend	28
Tablas en DB	49
Agentes IA	4 (Orchestrator, Reliability, Planning, Spare Parts)
Tool Wrappers	43 modulos, 126+ herramientas
Paginas frontend	25
Componentes UI	28 (8 custom + 20 shadcn/ui)
Funciones API client	98
Roles de usuario	5 (admin, manager, planner, tecnico, engineer)
Idiomas soportados	3 (es, en, ar)
Contenedores Docker	3 (backend, frontend, nginx)